PANDAS JOINS

Practice Assignment • Real-World DataFrame Operations

14 Questions	4 DataFrames	8 Join Concepts

Instructions

• Create the four DataFrames using the code provided below.

• Solve the questions using Pandas. Use **pd.merge()** wherever applicable.

• Use the specified join type and parameters for each question.

• For Anti Joins, use **indicator=True** or an appropriate **isin()** approach.

• Handle missing values where required and display only the requested columns.

• Do not modify the original DataFrames.

Part 1 – Create the DataFrames

Run the following code first. It creates all four datasets required for this assignment.

```
import pandas as pd
import numpy as np

# Table 1: Customers
customers = pd.DataFrame({
    "customer_id": [101, 102, 103, 104, 105],
    "customer_name": ["Rahul", "Priya", "Amit", "Neha", "Karan"],
    "city": ["Jodhpur", "Jaipur", "Delhi", "Mumbai", "Pune"]
})

# Table 2: Orders
orders = pd.DataFrame({
    "order_id": [1001, 1002, 1003, 1004, 1005],
    "customer_id": [101, 102, 101, 106, 103],
    "amount": [2500, 1800, 3200, 4500, 1200]
})

# Table 3: Products
products = pd.DataFrame({
    "product_id": ["P1", "P2", "P3", "P4"],
    "product_name": ["Laptop", "Mouse", "Keyboard", "Monitor"],
    "price": [50000, 800, 1500, 12000]
})

# Table 4: Employees
employees = pd.DataFrame({
    "employee_id": [1, 2, 3, 4, 5],
    "employee_name": ["Anil", "Sneha", "Vikram", "Pooja", "Mohit"],
    "manager_id": [np.nan, 1, 1, 2, 2]
})

# Display all DataFrames
print(customers)
print(orders)
print(products)
print(employees)
```

Dataset Reference

Use these column names when writing your merge conditions and selecting output columns.

Customers

customer_id	customer_name	city
101, 102, 103, 104, 105	Rahul, Priya, Amit, Neha, Karan	Jodhpur, Jaipur, Delhi, Mumbai, Pune

Orders

order_id	customer_id	amount
1001, 1002, 1003, 1004, 1005	101, 102, 101, 106, 103	2500, 1800, 3200, 4500, 1200

Products

product_id	product_name	price
P1, P2, P3, P4	Laptop, Mouse, Keyboard, Monitor	50000, 800, 1500, 12000

Employees

employee_id	employee_name	manager_id
1, 2, 3, 4, 5	Anil, Sneha, Vikram, Pooja, Mohit	NaN, 1, 1, 2, 2

Part 2 – Practice Questions

Read each question carefully. The join type and the Pandas concepts to use are mentioned below every question.

Level 1 – Basic Joins

Q1. Inner Join – Customers and Orders

Perform an Inner Join between **customers** and **orders** to display customer names along with their order IDs and order amounts. Include only customers who have placed orders.

Join type / concept: Inner Join

What to use: pd.merge() with **on="customer_id"** and **how="inner"**.

Q2. Left Join – All Customers

Perform a Left Join between **customers** and **orders** to display all customers along with their order details, including customers who have never placed an order.

Join type / concept: Left Join

What to use: pd.merge() with **on="customer_id"** and **how="left"**. Observe the missing values (NaN).

Q3. Right Join – All Orders

Perform a Right Join between **customers** and **orders** to display all orders along with customer information, including orders whose customer details are missing.

Join type / concept: Right Join

What to use: pd.merge() with **on="customer_id"** and **how="right"**. Observe the missing customer details.

Q4. Full Outer Join – Complete Customer and Order Records

Perform a Full Outer Join between **customers** and **orders** to display all matching and non-matching records from both DataFrames.

Join type / concept: Full Outer Join

What to use: pd.merge() with **on="customer_id"** and **how="outer"**.

Level 2 – Intermediate Joins

Q5. Left Anti Join – Customers Without Orders

Find all customers who have never placed an order. Display **customer_id**, **customer_name**, and **city**.

Join type / concept: Left Anti Join

What to use: pd.merge() with **how="left"** and **indicator=True**, then filter **_merge == "left_only"**. Alternatively, use **isin()**.

Q6. Right Anti Join – Orders Without Matching Customers

Find all orders whose **customer_id** does not exist in the **customers** DataFrame. Display **order_id**, **customer_id**, and **amount**.

Join type / concept: Right Anti Join

What to use: pd.merge() with **how="right"** and **indicator=True**, then filter **_merge == "right_only"**. Alternatively, use **isin()**.

Q7. Cross Join – Customers and Products

Perform a Cross Join between **customers** and **products** to generate every possible customer-product combination. Display **customer_name**, **city**, **product_name**, and **price**.

Join type / concept: Cross Join

What to use: pd.merge() with **how="cross"**. No common key is required.

Q8. Self Join – Employee and Manager

Perform a Self Join on the **employees** DataFrame to display each employee along with their respective manager's name. Display **employee_id**, **employee_name**, and **manager_name**.

Join type / concept: Self Join

What to use: Merge **employees** with itself using **left_on="manager_id"**, **right_on="employee_id"**, and **how="left"**. Use **suffixes** or rename the output columns.

Level 3 – Advanced Joins

Q9. Inner Join + Grouping – Customers with Multiple Orders

Perform an Inner Join between **customers** and **orders** and identify customers who have placed more than one order. Display customer names, order IDs, and order amounts.

Join type / concept: Inner Join + GroupBy / value_counts

What to use: Use **pd.merge()**, then use **groupby()** or **value_counts()** to identify customer IDs with more than one order. Filter the merged result to those customers.

Q10. Left Join + GroupBy – Total Amount Spent by Each Customer

Join **customers** and **orders** to calculate the total amount spent by each customer. Display **customer_id**, **customer_name**, **city**, and **total_amount**. Include customers who have not placed any orders and show their total amount as 0.

Join type / concept: Left Join + GroupBy + Missing-Value Handling

What to use: Use **how="left"**, then **groupby()** with **sum()**. Use **fillna(0)** where required.

Q11. Inner Join + Sorting – Highest-Value Order

Join **customers** and **orders** to find the customer who has placed the highest-value single order. Display customer name, order ID, city, and amount.

Join type / concept: Inner Join + sort_values()

What to use: Use **how="inner"**, then **sort_values()** on **amount** in descending order. Use **head(1)** to get the highest-value order.

Q12. Multiple DataFrame Joins – Customer Order Information

Combine **customers** and **orders** to display complete customer order information. Display **customer_name**, **city**, **order_id**, and **amount**. Also identify orders that do not have matching customer information.

Join type / concept: Left Join / Multiple Merge + Anti Join

What to use: Use **pd.merge()** on **customer_id**. Use **indicator=True** or a separate Right Anti Join to identify unmatched orders.

Q13. Cross Join + Filtering – Products Above a Price Threshold

Perform a Cross Join between **customers** and **products**. Then filter the result to display only products whose price is greater than ■1,000. Display customer name, product name, and price.

Join type / concept: Cross Join + Boolean Filtering

What to use: Use **how="cross"**, then filter rows using **price > 1000**.

Q14. Self Join + Sorting – Employee Hierarchy

Using the **employees** DataFrame, perform a Self Join to identify employees working under each manager. Display **manager_name** and **employee_name**. Sort the result by manager name.

Join type / concept: Self Join + sort_values()

What to use: Merge **employees** with itself using **left_on="manager_id"** and **right_on="employee_id"**. Filter out rows without a manager if the question asks only for employees under a manager, then use **sort_values()**.

Part 3 – Quick Join Reference

Question	Join / Concept	Main Pandas Tool
Q1	Inner Join	how="inner"
Q2	Left Join	how="left"
Q3	Right Join	how="right"

Question	Join / Concept	Main Pandas Tool
Q4	Full Outer Join	how="outer"
Q5	Left Anti Join	indicator=True + left_only
Q6	Right Anti Join	indicator=True + right_only
Q7	Cross Join	how="cross"
Q8	Self Join	Merge the DataFrame with itself
Q9	Inner Join + Grouping	groupby() / value_counts()
Q10	Left Join + GroupBy	groupby() + fillna()
Q11	Inner Join + Sorting	sort_values() + head(1)
Q12	Multiple Merge + Anti Join	pd.merge() + indicator=True
Q13	Cross Join + Filtering	Boolean condition
Q14	Self Join + Sorting	pd.merge() + sort_values()

Important Note: Pandas provides built-in merge modes for inner, left, right, outer, and cross joins. Anti Joins can be created by filtering merge results with **indicator=True** or by using **isin()**. Self Join is performed by merging a DataFrame with itself.

End of Assignment